COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010/DATA STRUCTURE AND ALGORITHM	**SECTION:**	CPE21S2
DATE PERFORMED:	09/14/26	**DATE SUBMITTED:**	09/16/26
NAME:	Bungag, Marc Jhowell M.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HOA 8.1 **Sort Techniques Part 2**

1. PROCEDURE OUTPUT

What procedures did you perform?

TABLE 8.1

CODE

main.cpp > main()

```cpp
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sortingAlgo.h"

const int maxSize = 100;

void displayArr(int arr[]);

int main() {
    int unsortedArr[maxSize];
    srand(time(0));
    for (int i = 0; i < maxSize; i++) {
        unsortedArr[i] = rand() % 100;
    }

    std::cout << "UnsortedArr: \n";
    displayArr(unsortedArr);
```

OBSERVATIONS

The program creates an array of 100 integers and fills it with random values from 0 to 99 using rand() % 100. The srand(time(0)) function makes the random values different each time the program is run. After generating the values, the program displays the unsorted array before applying any sorting algorithm.

TABLE 8.2

CODE

```cpp
// SHELL SORT
template <typename T>
void shellSort(T data[], size_t size) {
    size_t step = size / 2;

    while (step > 0) {
        for (size_t current = step; current < size; current++) {
            T value = data[current];
            int position = current;

            while (position >= (int)step &&
                   data[position - step] > value) {
                data[position] = data[position - step];
                position -= step;
            }
            data[position] = value;
        }
        step /= 2;
    }
}
```

```cpp
std::cout << "\n Shell Sort: \n";
shellSort(unsortedArr, maxSize);
displayArr(unsortedArr);
```

```
UnsortedArr:
28 93 28 44 49 15 59 75 0 29 11 3 50 32 62 27 85 35 56 65 76 13 36 91 7 76 74 52 6 70 82 71 91 79 7 96 58 58
 93 45 52 57 47 43 25 86 34 85 70 32 42 31 55 49 98 47 66 53 70 64 72 98 86 64 8 70 82 36 9 30 16 49 43 84 8
9 69 57 79 24 78 25 78 8 98 60 2 72 60 40 24 96 58 19 94 37 8 62 4 57 72

 Shell Sort:
0 2 3 4 6 7 7 8 8 8 9 11 13 15 16 19 24 24 25 25 27 28 28 29 30 31 32 32 34 35 36 36 37 40 42 43 43 44 45 47
 47 49 49 49 50 52 52 53 55 56 57 57 57 58 58 58 59 60 60 62 62 64 64 65 66 69 70 70 70 70 71 72 72 72 74 75
 76 76 78 78 79 79 82 82 84 85 85 86 86 89 91 91 93 93 94 96 96 98 98 98
```

OBSERVATIONS

The Shell Sort successfully rearranges the randomly generated values from the unsorted array into ascending order. It first compares elements that are separated by a larger gap, then gradually reduces the gap until it becomes 1, allowing the remaining elements to be sorted properly. After the sorting process, the displayed array starts from the smallest value and continues to the largest value. Shell Sort has a best-case time complexity of O(n log n) and can reach $O(n^2)$ in the worst case, depending on the gap sequence used.

TABLE 8.3

CODE

```cpp
// MERGE SORT
template <typename T>
void combine(T data[], int start, int mid, int end) {
    T sorted[100];

    int first = start;
    int second = mid + 1;
    int index = 0;

    while (first <= mid || second <= end) {

        if (second > end ||
            (first <= mid && data[first] <= data[second])) {
            sorted[index] = data[first];
            first++;
        }
        else {
            sorted[index] = data[second];
            second++;
        }
        index++;
    }

    for (int x = 0; x < index; x++) {
        data[start + x] = sorted[x];
    }
}

```

```cpp
std::cout << "\n Merge Sort: \n";
mergeSort(unsortedArr, 0, maxSize - 1);
displayArr(unsortedArr);
```

```
UnsortedArr:
61 32 69 71 29 37 82 7 84 19 17 80 83 55 34 12 64 16 68 31 28 59 61 67 68 60 96 71 28 44 10 0 36 81 2 4 89 5
 86 85 2 0 87 6 15 72 41 61 37 77 75 85 56 96 75 16 61 72 29 23 25 40 8 1 38 96 30 58 38 46 17 33 13 82 10 8
8 26 93 61 77 38 67 61 72 53 27 5 68 1 55 59 35 0 68 80 83 4 23 32 89

 Merge Sort:
0 0 0 1 1 2 2 4 4 5 5 6 7 8 10 10 12 13 15 16 16 17 17 19 23 23 25 26 27 28 28 29 29 30 31 32 32 33 34 35 36
 37 37 38 38 38 40 41 44 46 53 55 55 56 58 59 59 60 61 61 61 61 61 61 64 67 67 68 68 68 68 69 71 71 72 72 72
 75 75 77 77 80 80 81 82 82 83 83 84 85 85 86 87 88 89 89 93 96 96 96
```

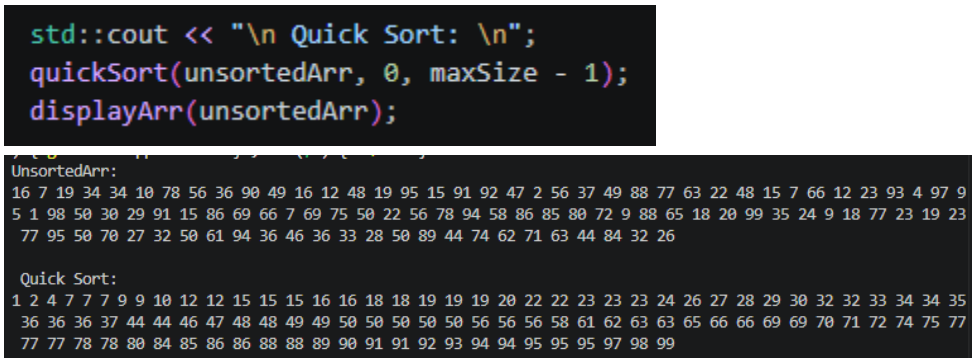
OBSERVATIONS

The Merge Sort successfully arranges the randomly generated numbers in ascending order. The algorithm divides the array into smaller sections, sorts each section, and then combines them back together in the correct order. After the process is completed, the output shows that the values are properly arranged from the smallest number to the largest number. Merge Sort has O(n log n) time complexity in the best, average, and worst cases because it consistently divides the array and merges the sorted parts.

TABLE 8.4

CODE

```
71  // QUICK SORT
72  template <typename T>
73  int split(T data[], int first, int last) {
74
75      T pivot = data[first];
76      int left = first + 1;
77      int right = last;
78
79      while (left <= right) {
80
81          while (left <= last && data[left] <= pivot)
82              left++;
83
84          while (data[right] > pivot)
85              right--;
86
87          if (left < right) {
88              std::swap(data[left], data[right]);
89          }
90      }
91
92      std::swap(data[first], data[right]);
93
94      return right;
95  }
96
97  template <typename T>
98  void quickSort(T data[], int first, int last) {
99      if (first >= last)
100         return;
101
102     int position = split(data, first, last);
103
104     quickSort(data, first, position - 1);
105     quickSort(data, position + 1, last);
106 }
107
108 #endif
```

	
OBSERVATIONS	The Quick Sort successfully sorts the randomly generated values in ascending order. The algorithm selects a pivot, separates the smaller and larger values around it, and then repeats the same process on the divided sections. After sorting is finished, the output shows that the numbers are arranged from the smallest value to the largest value. Quick Sort has O(n log n) in the best and average cases, but its worst case is O(n²) when the chosen pivot repeatedly creates very unbalanced partitions.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

PROBLEM #1

-Yes, we can sort the left and right sublists from the partition method using other sorting algorithms, such as Insertion Sort. For example, after Quick Sort divides the array into a left sublist and a right sublist, we can use Insertion Sort to arrange the elements in each sublist. This creates a combination of two sorting methods, where Quick Sort handles the partitioning while Insertion Sort completes the sorting of the smaller sections.

PROBLEM #2

CODE

suppsortingAlgo.h > SUPPSORTINGALGO_H

```h
#ifndef SUPPSORTINGALGO_H
#define SUPPSORTINGALGO_H
#include <iostream>

// MERGE SORT
template <typename T>
void merge(T arr[], int left, int mid, int right) {
    T temp[100];

    int i = left;
    int j = mid + 1;
    int k = 0;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp[k] = arr[i];
            i++;
        }
        else {
            temp[k] = arr[j];
            j++;
        }
        k++;
    }

    while (i <= mid) {
        temp[k] = arr[i];
        i++;
        k++;
    }

    while (j <= right) {
        temp[k] = arr[j];
        j++;
        k++;
    }

    for (i = left, k = 0; i <= right; i++, k++) {
        arr[i] = temp[k];
    }
}

```

```cpp
template <typename T>
void mergeSort(T arr[], int left, int right) {
    if (left < right) {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);

        merge(arr, left, mid, right);
    }
}


// QUICK SORT
template <typename T>
int partition(T arr[], int low, int high) {
    T pivot = arr[low];

    int i = low + 1;
    int j = high;

    while (i <= j) {

        while (i <= high && arr[i] <= pivot)
            i++;

        while (arr[j] > pivot)
            j--;

        if (i < j) {
            std::swap(arr[i], arr[j]);
        }
    }

    std::swap(arr[low], arr[j]);

    return j;
}

```

```
82  template <typename T>
83  void quickSort(T arr[], int low, int high) {
84      if (low < high) {
85
86          int position = partition(arr, low, high);
87
88          quickSort(arr, low, position - 1);
89          quickSort(arr, position + 1, high);
90      }
91  }
92
93  #endif
```

suppmain.cpp > main()

```cpp
#include <iostream>
#include "suppsortingAlgo.h"

void displayArr(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
    std::cout << std::endl;
}

int main() {

    int numbers[] = {
        4, 34, 29, 48, 53,
        87, 12, 30, 44, 25,
        93, 67, 43, 19, 74
    };

    int size = 15;

    std::cout << "Original Array:\n";
    displayArr(numbers, size);

    // Merge Sort
    mergeSort(numbers, 0, size - 1);

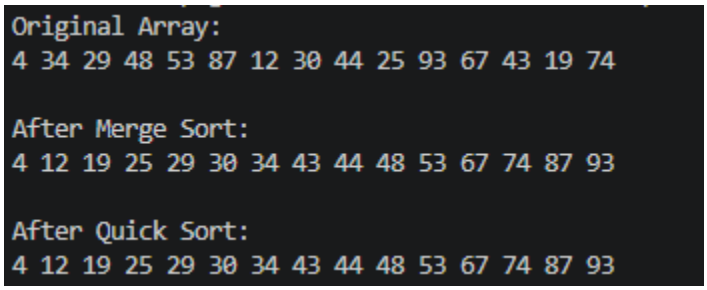
    std::cout << "\nAfter Merge Sort:\n";
    displayArr(numbers, size);

    // Reset the array
    int numbers2[] = {
        4, 34, 29, 48, 53,
        87, 12, 30, 44, 25,
        93, 67, 43, 19, 74
    };

    // Quick Sort
    quickSort(numbers2, 0, size - 1);

    std::cout << "\nAfter Quick Sort:\n";
    displayArr(numbers2, size);

    return 0;
}
```

	
OBSERVATIONS	For the given array {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}, Merge Sort and Quick Sort can both be used to sort the values efficiently. Merge Sort divides the array into smaller parts and combines them in sorted order, while Quick Sort uses a pivot to divide the values into smaller and larger groups. Both have an average time complexity of O(n log n), which makes them efficient for sorting larger amounts of data. For the given array, Quick Sort can provide the fastest time performance in practice because it usually has fewer extra operations and memory use than Merge Sort. Its average time complexity is O(n log n), although its worst case can be $O(n^2)$ depending on the pivot selection.

CONCLUSION:
Overall, this activity helped me understand how different sorting algorithms work and how their time complexities affect their performance. I learned that Shell Sort, Merge Sort, and Quick Sort use different methods to arrange data, with Merge Sort and Quick Sort generally providing efficient performance for larger data sets. I also learned that the choice of algorithm can depend on the size and arrangement of the data, since each algorithm has its own advantages and limitations.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance

with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.